QuantumRisk: Hybrid Quantum ML Patient Risk Prediction

Capstone 23CSE463, Problem Statement 28. Design a Quantum Machine Learning-based system to predict patient health risks from clinical data.

Implementation plan for building the prototype in Google Antigravity.

1. The Solution in One Paragraph

QuantumRisk takes a patient's clinical features, compresses them to a small set of the most informative variables, encodes them into a parameterised quantum circuit, and outputs a calibrated risk probability (Low / Medium / High). The same circuit is benchmarked against classical models (Logistic Regression, SVM, Random Forest, XGBoost, MLP). A 3D web interface lets users watch the patient's data become a quantum state: one Bloch sphere per qubit, glowing entanglement links between qubits, and a probability landscape over all basis states, ending in a risk gauge.

What makes it capstone-worthy

- Three quantum models compared head-to-head: a Variational Quantum Classifier (VQC), a Quantum Kernel SVM (QSVM), and a Hybrid Classical-Quantum neural network.
- Honest benchmarking against classical baselines with proper cross-validation, calibration and noise robustness.
- Quantum-native explainability: per-qubit purity and entanglement, which classical models cannot show.
- A polished 3D prototype that makes the quantum mechanics visible.

Honest framing (important for viva and report). Do not claim quantum advantage. Clinical tabular datasets are small, and simulators run the circuits. The defensible claims are: (a) comparable accuracy with far fewer trainable parameters, (b) the kernel and variational methods are a valid NISQ-era approach, (c) the pipeline is hardware-ready (swap the simulator for IBM Quantum). State clearly this is a decision-support research prototype, not a medical device.

2. Data and Problem Framing

Item	Choice
Primary dataset	UCI Heart Disease (Cleveland, 303 rows, 13 features), binary target: disease present / absent
Secondary dataset	Pima Indians Diabetes (768 rows, 8 features), binary target: diabetic / not
Why these	Small, public, de-identified, few features, so they fit on 4 to 8 qubits
Multi-risk mode	One trained model per condition; the UI has a condition selector (Heart / Diabetes)
Split	Stratified 5-fold cross-validation, plus a held-out 20% test set never touched during tuning
Privacy	Public de-identified data only; no real patient data is stored; inference requests are not persisted by default

Preprocessing pipeline

1. Impute missing values (median for numeric, mode for categorical). Pima has hidden zeros in glucose, BP, BMI that must be treated as missing.
2. One-hot encode categoricals, then standardise.
3. **Feature reduction to n features (n = 4, 6, 8):** mutual information ranking, with PCA as an ablation. This is required because qubit count is limited.
4. Rescale to [0, pi] for rotation encoding.
5. Handle class imbalance with class weights (not SMOTE inside the CV folds, to avoid leakage).

3. Quantum Model Architecture

3.1 Overall hybrid pipeline

```
Clinical features (13)
   -> Impute / encode / scale
   -> Feature selection (top n = 4..8)
   -> Rescale to [0, pi]
   -> QUANTUM FEATURE MAP  (data -> quantum state)
   -> VARIATIONAL ANSATZ   (trainable rotations + entanglers)
   -> MEASUREMENT          (expectation of Z / parity)
   -> Sigmoid + Platt calibration
   -> Risk probability -> Low / Medium / High
        ^
        |  classical optimiser updates circuit parameters
        +--- loss (binary cross-entropy) <--- labels
```

3.2 Model A: Variational Quantum Classifier (main model)

- **Qubits:** n = 4 (prototype), 6 and 8 for scaling study.
- **Feature map:** `ZZFeatureMap(feature_dimension=n, reps=2, entanglement='linear')`. Each feature goes into one qubit through H plus phase rotation, and pairwise ZZ interactions encode feature correlations.
- **Ansatz:** `RealAmplitudes(n, reps=3)` or `EfficientSU2(n, reps=2)`. Rotation layers (RY, RZ) alternate with CNOT entanglers. Parameters: about 16 to 48.
- **Readout:** parity of measured bitstring (even = low risk, odd = high risk), or expectation value of Z on qubit 0 mapped through a sigmoid. Use the Estimator primitive for exact expectations, Sampler for shot-based.
- **Training:** loss is binary cross-entropy, optimiser is COBYLA (gradient-free baseline) then SPSA or Adam with parameter-shift gradients. Early stopping on validation AUROC.
- **Libraries:** `qiskit`, `qiskit-machine-learning` (VQC, TorchConnector), `qiskit-aer`.

3.3 Model B: Quantum Kernel SVM (QSVM)

- Kernel is the fidelity `K(x, x') = |<phi(x)|phi(x')>|^2` with a `FidelityQuantumKernel` using the same ZZ feature map.
- Feed the kernel matrix to a classical `SVC(kernel='precomputed')`.
- No variational training, so it is a strong, stable quantum baseline. Cost is O(N^2) kernel evaluations, which is fine for 300 to 800 rows.
- Tune the feature-map reps and a bandwidth scaling factor on the inputs.

3.4 Model C: Hybrid Classical-Quantum Neural Network

```
Input (13) -> Dense(16, ReLU) -> Dense(n) -> tanh * pi
          -> Quantum layer (n qubits, AngleEmbedding +
StronglyEntanglingLayers)
          -> Dense(1) -> Sigmoid
```

- Built with **PennyLane + PyTorch** (`qml.qnn.TorchLayer`), trained end-to-end with Adam.
- Lets the classical layers learn the compression, so the quantum layer sees learned features rather than hand-picked ones.

3.5 Classical baselines (mandatory)

Logistic Regression, SVM (RBF), Random Forest, XGBoost, and a small MLP with a similar parameter count to the quantum model. Use the same splits and preprocessing for every model.

3.6 Evaluation protocol

Dimension	Metrics
Accuracy	AUROC (primary), accuracy, F1
Clinical safety	Sensitivity (recall of the positive class) and specificity at a chosen threshold; confusion matrix
Reliability	Brier score, calibration curve
Efficiency	Trainable parameters, circuit depth, qubits, training time, inference latency
Robustness	Performance under Aer noise models (depolarising, readout error) at 0%, 1%, 2%, 5% error; shot count sweep (256, 1024, 4096)
Scaling	AUROC vs number of qubits (4, 6, 8) and ansatz depth (1 to 4)
Statistics	Mean and standard deviation over 5 folds and 3 random seeds

3.7 Quantum-native explainability

1. **Feature attribution:** permutation importance and leave-one-feature-out on the quantum model, plus SHAP (KernelExplainer) on the full predict function.

2. **Per-qubit purity** `Tr(rho_i^2)`: low purity means that qubit is strongly entangled with the rest.
3. **Entanglement map:** pairwise mutual information (or concurrence) between qubits for each patient. This drives the 3D entanglement links.
4. **Bloch vectors** `(x, y, z)` from reduced density matrices (the partial-trace technique from your course), used directly by the 3D UI.

4. Full-Stack Architecture

```
+---------------------------- FRONTEND (Next.js) ---------------------------+
|  Landing | Patient Intake | Quantum Lab (3D) | Compare | Noise Lab | Ethics
|
|  React Three Fiber scenes + Recharts + Zustand state + framer-motion      |
+-------------------------------+-----------------------------------------+
                                | REST (JSON)  +  WebSocket (training logs)
+-------------------------------v-----------------------------------------+
|                       BACKEND (FastAPI, Python 3.11)
|
|  /predict  /compare  /noise  /train  /explain  /metrics  /health          |
|  Services: preprocess | quantum_models | baselines | explain | state_viz
|
+-------------------------------+-----------------------------------------+
                                |
+-------------------------------v-----------------------------------------+
|  QUANTUM / ML ENGINE
|
|  Qiskit (Aer statevector + noisy sim) | PennyLane | scikit-learn | XGBoost
|
|  Optional: IBM Quantum Runtime backend (flag-switched, off by default)
|
+-------------------------------+-----------------------------------------+
                                |
+-------------------------------v-----------------------------------------+
|  STORAGE: /artifacts (joblib models, params.json, metrics.json)           |
|           SQLite (optional: experiment runs, no patient PII)
|
+---------------------------------------------------------------------------+
```

Key design decision: training happens offline in notebooks or scripts and saves artifacts. The API only loads trained parameters and runs fast inference, so the demo never waits on training. The `/train` endpoint exists for the live "watch it learn" demo with a small number of iterations.

4.1 Backend

Stack: FastAPI, Pydantic v2, Uvicorn, Qiskit, qiskit-aer, qiskit-machine-learning, PennyLane, torch (CPU), scikit-learn, xgboost, shap, joblib, numpy, pandas.

Endpoints

Method	Path	Purpose
GET	`/health`	Liveness and loaded-model info
GET	`/schema/{condition}`	Feature names, ranges, units for the intake form
POST	`/predict`	Patient features in; risk plus quantum state data out
POST	`/compare`	Same patient through all quantum and classical models
POST	`/explain`	Feature attributions plus per-qubit analysis
POST	`/noise`	Re-run prediction at a given noise level and shot count
GET	`/metrics/{condition}`	Stored CV benchmark results for charts
WS	`/ws/train`	Streams loss and AUROC per iteration for a short live training run

`/predict` response contract (what the 3D UI consumes)

```
{
  "condition": "heart",
  "risk_probability": 0.73,
  "risk_level": "High",
  "model": "vqc",
  "n_qubits": 4,
  "features_used": ["thal", "ca", "cp", "oldpeak"],
  "encoded_angles": [2.41, 0.0, 1.57, 0.88],
  "circuit": [
    {"layer": 0, "gate": "H",  "qubits": [0]},
    {"layer": 1, "gate": "RZ", "qubits": [0], "param": 2.41},
    {"layer": 2, "gate": "CX", "qubits": [0, 1]}
  ],
  "qubit_states": [
    {"qubit": 0, "bloch": [0.31, -0.12, 0.94], "purity": 0.88},
    {"qubit": 1, "bloch": [0.05, 0.40, 0.55], "purity": 0.47}
  ],
```

```
  "entanglement": [[0, 0.62, 0.1, 0.0], [0.62, 0, 0.3, 0.1]],
  "state_probabilities": {"0000": 0.04, "0001": 0.11},
  "explanation": {"thal": 0.31, "ca": 0.24, "cp": 0.19, "oldpeak": 0.12},
  "baselines": {"logreg": 0.69, "rf": 0.77, "xgb": 0.75},
  "disclaimer": "Research prototype. Not a medical device."
}
```

Backend implementation notes

- Compute Bloch vectors by simulating the statevector of the circuit (n is at most 8, so it is instant), then `partial_trace` for each qubit and extract `(Tr(rho X), Tr(rho Y), Tr(rho Z))`.
- Compute entanglement via mutual information from the two-qubit reduced density matrices using von Neumann entropy.
- Cache loaded models at startup. Enable CORS for the frontend origin only.
- Input validation with Pydantic ranges (for example age 1 to 120) and return clear 422 errors.
- Add a small `pytest` suite: statevector norm equals 1, Bloch vector length is at most 1, risk is in [0, 1], and deterministic output with a fixed seed.

4.2 Frontend

Stack: Next.js 14 (App Router), TypeScript, Tailwind CSS, shadcn/ui, **three.js with React Three Fiber and drei**, `@react-three/postprocessing` (bloom), framer-motion, Zustand, Recharts, TanStack Query.

Visual identity: dark "quantum lab" theme (near-black navy), neon cyan, violet and magenta accents, soft bloom glow, subtle star-field or particle background, monospaced numerals for readouts.

Pages

1. **Landing:** animated 3D hero (rotating Bloch sphere with orbiting state vector), one-line pitch, "Launch Quantum Lab".
2. **Patient Intake:** clean form generated from `/schema`, sliders and selects, a "Load sample patient" button (low / medium / high examples) for demos.
3. **Quantum Lab (the showpiece):** the 3D scene described below plus a side panel with the risk gauge and explanation.
4. **Model Comparison:** bar and ROC charts for VQC vs QSVM vs Hybrid vs classical baselines, with parameter counts and training times.
5. **Noise Lab:** sliders for noise level and shots; live chart of how risk and accuracy degrade.
6. **Training Theatre:** live loss curve over WebSocket while parameters move on the circuit.

7. **Ethics and Limits:** dataset provenance, limitations, no-medical-advice statement.

3D scene components (Quantum Lab)

Component	What it shows	Data source
`BlochSphere` x n	One translucent sphere per qubit with axes, equator ring and a glowing state arrow. Arrow length = purity, so entangled qubits visibly shrink inside the sphere.	`qubit_states`
`FeatureLabel`	Each sphere is labelled and coloured by the clinical feature it encodes (for example "Cholesterol")	`features_used`
`EntanglementBeams`	Animated arcs between spheres; thickness and brightness scale with mutual information	`entanglement`
`CircuitTimeline`	Gates as 3D tiles on horizontal wires; a light pulse travels left to right as the circuit "executes"	`circuit`
`ProbabilityLandscape`	2^n glowing bars (16 for 4 qubits) whose heights are basis-state probabilities; bars of high-risk parity coloured magenta, low-risk cyan	`state_probabilities`
`RiskCore`	A pulsing energy orb or gauge: colour and pulse speed map to risk level; number counts up to the probability	`risk_probability`
`CameraRig`	Smooth guided tour (Intake, Encode, Entangle, Measure, Result) with a step indicator	UI state

UX flow: submit patient, a 4-step guided animation plays (encode angles into spheres, run circuit gates, entanglement beams appear, measurement collapses the landscape into the risk core). A "skip animation" button and "free orbit" mode are included. Respect `prefers-reduced-motion`.

Performance and fallback: cap devicePixelRatio at 2, use instancing for bars, lazy-load the 3D canvas, and provide a 2D chart fallback if WebGL is unavailable.

4.3 Repository structure

```
quantumrisk/
  README.md
  docker-compose.yml
  backend/
    app/
      main.py
      api/ (predict.py, compare.py, explain.py, noise.py, train_ws.py)
      core/ (config.py, schemas.py)
      services/
        preprocess.py
        quantum_vqc.py
        quantum_kernel.py
        hybrid_qnn.py
        baselines.py
        state_viz.py        # statevector, partial trace, Bloch, entanglement
        explain.py
        noise.py
    ml/
      data/ (heart.csv, pima.csv)
      notebooks/ (01_eda, 02_baselines, 03_vqc, 04_qsvm, 05_hybrid, 06_noise)
      train.py
      artifacts/
    tests/
  frontend/
    app/ (page.tsx, intake/, lab/, compare/, noise/, training/, ethics/)
    components/
      three/ (BlochSphere, EntanglementBeams, CircuitTimeline,
              ProbabilityLandscape, RiskCore, CameraRig, Scene)
      ui/ (shadcn)
      charts/ (RocChart, MetricsBar, NoiseChart, LossCurve)
    lib/ (api.ts, store.ts, types.ts)
  docs/ (architecture.md, report.md, demo-script.md)
```

5. Phased Implementation Plan

Timeline is expressed in working days; compress or stretch to your deadline. **Build the backend contract first, because the 3D UI is only a renderer of it.**

Phase	Days	Deliverable	Done when
0. Setup	0.5	Repo, venv, `pip install qiskit qiskit-aer qiskit-machine-learning pennylane torch scikit-learn xgboost fastapi shap`; Next.js app scaffolded	Both servers run hello-world
1. Data and baselines	1.5	Cleaned datasets, feature ranking, 5 classical baselines with CV metrics saved to `metrics.json`	Baseline table exists
2. Quantum models	3	VQC (4 qubits) trained and saved; QSVM; Hybrid QNN; ablations on qubits and depth	VQC AUROC within a few points of baselines
3. State and explain services	1	`state_viz.py` outputs Bloch, purity, entanglement; permutation importance	Unit tests pass
4. API	1	All endpoints with Pydantic schemas, WebSocket training stream	Swagger UI returns valid JSON
5. Frontend foundation	1.5	Layout, theme, intake form, API client, charts pages	Form submit shows raw result
6. 3D Quantum Lab	3	Bloch spheres, beams, circuit timeline, probability landscape, risk core, guided camera	Full animated flow works from live API data
7. Noise and training theatre	1	Noise Lab and live training page	Sliders change results in real time
8. Polish and deploy	1.5	Responsive layout, loading states, disclaimer, Docker, deploy	Public URL works
9. Report and demo	1.5	Report, slides, 3-minute demo script, results tables	Rehearsed demo

Suggested deployment: frontend on Vercel; backend as a Docker container on Render, Railway or Hugging Face Spaces (CPU is enough). Keep a local `docker-compose up` fallback for the viva.

6. Working in Google Antigravity

Use the **Manager view** to run parallel agents, and give each one a bounded task and a testable acceptance criterion. Ask every agent to produce an **Artifact** (plan, test results, screenshot or recording) before it merges.

Agent 1: Data and ML

> Build `backend/ml/` for the UCI Heart Disease dataset: cleaning, mutual-information feature selection (n = 4, 6, 8), stratified 5-fold CV, and baselines (LogReg, SVM, RF, XGBoost, MLP). Save metrics.json with AUROC, F1, sensitivity, specificity and Brier score. Then implement a Qiskit VQC (ZZFeatureMap reps=2 plus RealAmplitudes reps=3, COBYLA then SPSA), a FidelityQuantumKernel SVC, and a PennyLane TorchLayer hybrid. Save artifacts. Acceptance: reproducible with `python train.py --seed 42`, results table printed.

Agent 2: API and quantum state services

> Build the FastAPI app in `backend/app/` using the response contract in the plan. Implement `state_viz.py` to compute statevector, per-qubit reduced density matrices via partial trace, Bloch vectors, purities and pairwise mutual information. Add pytest tests (norm, Bloch length at most 1, risk in [0, 1]). Acceptance: `/predict` returns the full JSON for three sample patients.

Agent 3: Frontend and 3D

> Build the Next.js 14 TypeScript app with a dark neon quantum theme. Implement the Quantum Lab with React Three Fiber: BlochSphere per qubit (arrow length equals purity), EntanglementBeams, CircuitTimeline with a travelling pulse, ProbabilityLandscape (instanced bars), RiskCore, guided CameraRig, bloom postprocessing. Consume the `/predict` contract; mock it with a JSON fixture until the backend is ready. Acceptance: browser recording of the full guided flow at 60fps on a laptop, with a 2D fallback.

Agent 4 (later): QA and docs

> Write README, architecture doc, and a demo script. Run an end-to-end test through the browser and report failures.

Tips

- Freeze the JSON contract before agents start; changes go through you.

- Keep n = 4 qubits for the demo (16 bars is readable), and show 6 and 8 only in the scaling charts.
- Ask for the mock-data path early so frontend and backend never block each other.

7. Risks and Mitigations

Risk	Mitigation
Quantum model underperforms classical	Report honestly; emphasise parameter efficiency, kernel method, noise study, and explainability; tune feature map and bandwidth
Slow simulation	Keep n at most 8, use statevector (exact) for inference, cache kernels, precompute training offline
Overfitting on 303 rows	Strict CV, held-out test set, few parameters, report standard deviation over seeds
Data leakage	Fit scaler and feature selector inside each fold only
3D complexity and time	Build one component at a time; mock data first; 2D fallback; keep scenes light
Ethical concerns	Public de-identified data, visible disclaimer, no storage of inputs, sensitivity-first threshold discussion

8. Report and Viva Outline

1. Problem and motivation (clinical risk screening, why small-qubit QML is relevant).
2. Background: feature maps, variational circuits, quantum kernels, NISQ limits.
3. System architecture and data pipeline.
4. Model design and hyperparameters.
5. Results: baseline vs quantum table, ablations, noise robustness, scaling.
6. Explainability with the entanglement and purity analysis.
7. Prototype walkthrough (screenshots of the 3D lab).
8. Limitations, ethics, and future work (real hardware via IBM Quantum, larger feature sets, federated learning, error mitigation).

3-minute demo script: (1) Pick a sample patient. (2) Watch features become qubit rotations. (3) Watch the circuit run and entanglement form. (4) See the measurement collapse into a risk score. (5) Open Compare to show quantum vs classical. (6) Drag the noise slider to show robustness. (7) Close with limitations and the path to real hardware.

9. Next Steps

1. Lock the dataset (Heart Disease first) and the `/predict` JSON contract.
2. Start Agent 1 and Agent 3 in parallel (the frontend works from a fixture).
3. Merge at Phase 6 and test end to end.